Bean Lifecycle

→ IoC container will create all the bean definitions defined for an application, and only then afterwards proceed to create actual beans

→ After bean definitions are created, and the IoC container is instantiating object, it can wire the dependencies instantly if the wiring is through constructor injection. Otherwise, dependency is injected later on

→ Methods which are invoked by Spring & not by the developer are called **callback methods**

Aware interface

```
@Component
public class UserService implements BeanNameAware {

    public UserService() { }

    @Override
    public void setBeanName(String name) {
        //...
    }
}
```

Callback method

→ Such callback methods don't have a lot of business logic impact. Rather, they could be used in a LOGGER functionality to log which bean caused what, from which application context has the bean / error come from etc.

Initialization Callbacks

→ In the bean lifecycle, there is a phase b/w object creation & it's actual use in the code.

1. **InitializingBean**

→ Implement this interface & override the `afterPropertiesSet()` method

2. **initMethod**

→ If you're created your bean definition inside a configuration class, you can create a custom method & pass the method name to the bean definition to call that method

```
public class CartService {
    private final Map<Integer, String> map;

    public CartService() {
        map = new HashMap<>();
    }

    public void preUse() {
        map.put(1, "Keshav");
        map.put(2, "Krishna");
    }

    //...
}
```

```
@Configuration
@ComponentScan
public class AppConfig {

    @Bean(initMethod = "preUse")
    public CartService createCartService() {
        return new CartService();
    }

    //...
}
```

3. PostConstruct annotation (Recommended)

→ Most widely used.

→ In @Component classes, just place @PostConstruct annotation on the method you want Spring to callback once your Object is ready

```
@PostConstruct
public void initializeBean() {
    System.out.println("Inside @PostConstruct method");
    map.put(1, "Jitu");
    map.put(2, "Neetu");
}
```

Why not use Constructors instead of @PostConstruct

→ Just because a constructor has been called, does not mean the Object is fully ready yet.

→ It might have field injections / setter injections for the dependencies which are not wired to the object yet.

→ There might be aware interface methods which have not been called yet.

→ So, we want to perform operations, only after the object creation is complete and is in ready state.

→ Also, there might be some expensive / heavy operations involved which we do not want to take up object creation's time itself.

Bean Lifecycle till NOW ?

IoC container started
↓
Read configuration
↓
Read Bean Definitions
↓
Instantiate Objects
↓
Wire Dependencies
↓
Aware interfaces are called
↓
Initialization Callbacks
↳ at this stage we should perform any initial operation, ∴ constructors for such activities is not recommended

Destruction Callbacks

→ Just like initialization callback, Spring provides us with a destruction callback function just before the IoC container is about to destroy the bean from the container

Bean Lifecycle continued...

↓

Initialization Callback

↓

Bean is Ready to Use

↓

Destruction Callbacks

↓

Bean is destroyed

→ In this phase, you might want to close certain connections / heavy operations, clean cache etc.

1. Disposable Bean

→ It's an aware interface which the bean class has to implement, override the `destroy()` method & perform cleanup operations there.

2. destroy Method

→ If you have created your bean definition via a configuration component, you can pass any custom cleanup method as the destroy method for that bean definition

```
@Bean(initMethod = "start", destroyMethod = "preDestroy")
//...
```

3. @PreDestroy (Recommended)

→ Exactly same usecase, most widely used.
→ Just mention this annotation on top of your cleanup method for the bean

```
@PreDestroy
public void destroy() {
    map.clear();
    System.out.println("@PreDestroy method called");
}
```

Lazy & Prototype Lifecycle

→ The lifecycle phases discussed above is for Singleton beans with by default eager scope.
→ Even if they are made @Lazy, the steps are all the same, just the difference is the bean will not be instantiated, callbacks made until & unless we use the bean in code somewhere

→ Now, we know that Prototype beans are lazy by default. So is there bean lifecycle phase same as Singleton beans with lazy scope?
⇒ Not exactly
  → We know that for every where a bean is needed, the IoC container provides a fresh bean
  → That's it
  → The lifecycle is handled by Spring until it's creation, i.e from
  ... → Bean Definition → Instantiate → Wire Deps. → Aware interfaces → Init callbacks → Bean is Ready to Use

→ Once the bean is handed over to the client / developer, Spring no longer starts managing it & hands the full control back

→ ∴, We can't use destroy callbacks on prototype beans

But like Why?

→ To avoid memory leak

→ Heap memory might keep getting filled with unused beans as they can't be garbage collected since they are inside / being managed by the IoC container

→ Hence, the prototype beans are destroyed by native garbage collection.

→ So, garbage collection will help clear the memory occupied by the prototype, but if it's an expensive resource which needs clearing / cleanup, the developer will have to handle it on it's own

↳ use java's try-with resources, finally